

JWT → Spring Security → Keycloak for Spring Boot Microservices

Architecture

```
Client
  ↓
API Gateway
  ↓ validates JWT
Student Service
Address Service
```

PART 1: JWT Authentication

Step 1: Create auth-service

Dependencies:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.7</version>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.7</version>
  <scope>runtime</scope>
```

```
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.7</version>
  <scope>runtime</scope>
</dependency>
```

JwtUtil

```
package com.univeristy.ms.security;

import java.security.Key;
import java.util.Date;

import javax.crypto.spec.SecretKeySpec;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;

public class JwtUtil {

    private static final String SECRET =
        "mysecretkeymysecretkeymysecretkey12";

    private static final Key KEY =
        new SecretKeySpec(SECRET.getBytes(),
            SignatureAlgorithm.HS256.getJcaName());

    public static String generateToken(String username) {

        return Jwts.builder()
            .subject(username)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis()
                + 1000 * 60 * 60))
            .signWith(KEY)
            .compact();
    }

    public static String extractUsername(String token) {
```

```

        Claims claims = Jwts.parser()
            .verifyWith((javax.crypto.SecretKey) KEY)
            .build()
            .parseSignedClaims(token)
            .getPayload();

        return claims.getSubject();
    }

    public static boolean validateToken(String token) {

        try {
            Jwts.parser()
                .verifyWith((javax.crypto.SecretKey) KEY)
                .build()
                .parseSignedClaims(token);

            return true;
        } catch (Exception ex) {
            return false;
        }
    }
}

```

Login Request

```

package com.univeristy.ms.request;

import lombok.Data;

@Data
public class LoginRequest {

    private String username;
    private String password;
}

```

Auth Controller

```
package com.univeristy.ms.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.univeristy.ms.request.LoginRequest;
import com.univeristy.ms.security.JwtUtil;

import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/auth")
public class AuthController {

    @PostMapping("/login")
    public ResponseEntity<?> login(
        @RequestBody LoginRequest request) {

        if (request.getUsername().equals("admin")
            && request.getPassword().equals("admin")) {

            String token =
                JwtUtil.generateToken(request.getUsername());

            Map<String, String> response = new HashMap<>();

            response.put("token", token);

            return ResponseEntity.ok(response);
        }

        return ResponseEntity.badRequest()
            .body("Invalid credentials");
    }
}
```

Test Login

```
POST http://localhost:8085/auth/login
```

Body:

```
{
  "username": "admin",
  "password": "admin"
}
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9..."
}
```

PART 2: API Gateway JWT Validation

Gateway Dependency

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway-server-webflux</artifactId>
</dependency>
```

JWT Gateway Filter

```
package com.univeristy.ms.filter;

import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
```

```

import com.univeristy.ms.security.JwtUtil;

import reactor.core.publisher.Mono;

@Component
public class JwtAuthenticationFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange,
                           GatewayFilterChain chain) {

        String path = exchange.getRequest()
            .getURI()
            .getPath();

        if (path.contains("/auth/login")) {
            return chain.filter(exchange);
        }

        String authHeader = exchange.getRequest()
            .getHeaders()
            .getFirst(HttpHeaders.AUTHORIZATION);

        if (authHeader == null
            || !authHeader.startsWith("Bearer ")) {

            exchange.getResponse()
                .setStatusCode(HttpStatus.UNAUTHORIZED);

            return exchange.getResponse().setComplete();
        }

        String token = authHeader.substring(7);

        if (!JwtUtil.validateToken(token)) {

            exchange.getResponse()
                .setStatusCode(HttpStatus.UNAUTHORIZED);

            return exchange.getResponse().setComplete();
        }

        return chain.filter(exchange);
    }
}

```

Call API Through Gateway

```
GET http://localhost:9090/student-service/api/student/getById/1
```

Header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

PART 3: Spring Security Resource Server

Add Dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

Security Config

```
package com.univeristy.ms.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(
        HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**")
                .permitAll())
    }
}
```

```
        .anyRequest()  
        .authenticated()  
        .oauth2ResourceServer(oauth2 -> oauth2.jwt());  
  
    return http.build();  
}  
}
```

PART 4: Keycloak Setup

Download Keycloak

Official site:

<https://www.keycloak.org/downloads>

Run Keycloak Docker

```
docker run -p 8081:8080  
-e KEYCLOAK_ADMIN=admin  
-e KEYCLOAK_ADMIN_PASSWORD=admin  
quay.io/keycloak/keycloak:26.2 start-dev
```

Open Keycloak

```
http://localhost:8081
```

Login:

```
admin/admin
```

Create Realm

Example:


```
microservice-realm
```

Create Client

```
student-client
```

Enable:

- Client Authentication = ON
 - Direct Access Grants = ON
-

Create User

Example:

```
username = vadiraj  
password = password
```

Get Token From Keycloak

```
POST http://localhost:8081/realms/microservice-realm/protocol/openid-connect/  
token
```

Form Data:

```
client_id=student-client  
client_secret=<secret>  
grant_type=password  
username=vadiraj  
password=password
```

Response:

```
{  
  "access_token": "eyJhbGciOi..."  
}
```

Configure Gateway with Keycloak

Dependency

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>  
</dependency>
```

application.properties

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=  
http://localhost:8081/realms/microservice-realm
```

Security Config

```
package com.univeristy.ms.config;  
  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.security.config.annotation.web.builders.HttpSecurity;  
import org.springframework.security.web.SecurityFilterChain;  
  
@Configuration  
public class SecurityConfig {  
  
    @Bean  
    public SecurityFilterChain securityFilterChain(  
        HttpSecurity http) throws Exception {  
  
        http  
            .csrf(csrf -> csrf.disable())  
            .authorizeHttpRequests(auth -> auth
```

```
        .requestMatchers("/auth/**")
        .permitAll()
        .anyRequest()
        .authenticated())
        .oauth2ResourceServer(oauth2 -> oauth2.jwt());

    return http.build();
}
}
```

Final Production Architecture

```
Client
  ↓
Keycloak Authentication
  ↓ returns JWT
API Gateway validates JWT
  ↓
Student Service
Address Service
```

Recommended Learning Order

1. Simple JWT token generation
2. Gateway JWT validation
3. Spring Security Resource Server
4. Keycloak integration
5. Roles and Authorities
6. Refresh Tokens
7. OAuth2 Authorization Code Flow

Useful URLs

Eureka

```
http://localhost:8761
```

Gateway

`http://localhost:9090`

Keycloak

`http://localhost:8081`

Swagger

`http://localhost:8080/swagger-ui/index.html`